

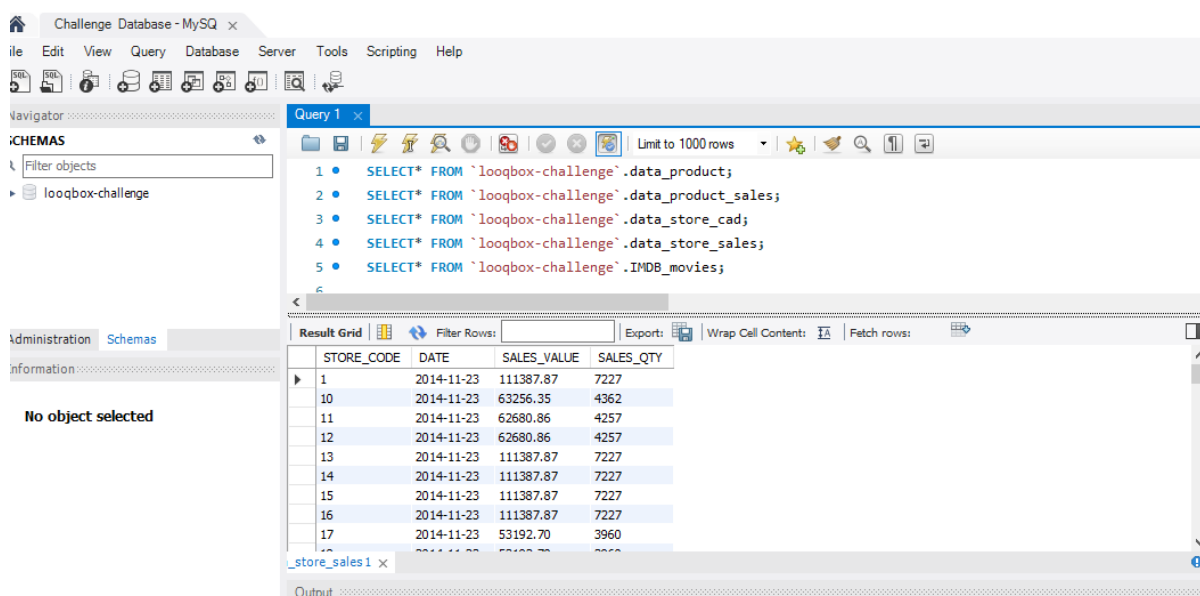
Desafio Técnico – Looqbox

Introdução

Este documento apresenta a resolução do desafio técnico proposto pela Looqbox, contemplando consultas SQL, desenvolvimento de funções em Python para recuperação de dados e criação de visualizações analíticas.

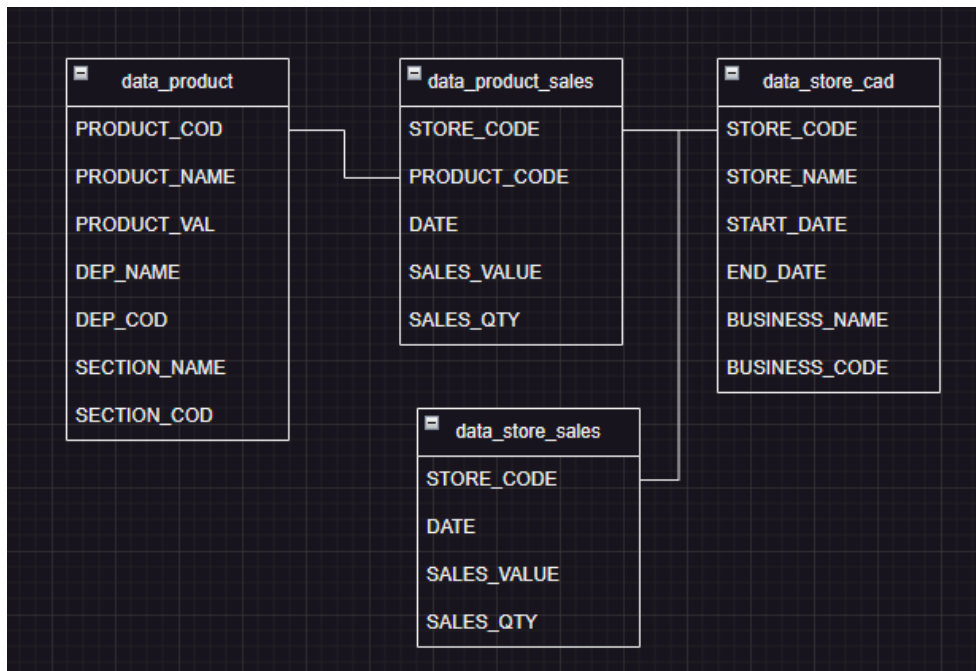
SQL Test

Antes de iniciar a resolução das questões, realizei a conexão com o banco de dados e explorei todas as tabelas utilizando consultas simples para compreender a estrutura dos dados, identificar as principais colunas e entender como as informações estavam distribuídas.



Após essa etapa exploratória, criei um diagrama simples utilizando a ferramenta Draw.io para mapear os possíveis relacionamentos entre as tabelas. O objetivo foi facilitar a compreensão do modelo de dados, identificar as chaves utilizadas nos joins

e servir como apoio durante a resolução das consultas SQL e dos cases propostos.



Questão 1

Pergunta: What are the 10 most expensive products in the company?

Para responder essa pergunta, consultei a tabela `data_product`, que contém as informações dos produtos. Selecionei as colunas `PRODUCT_NAME` e `PRODUCT_VAL`, responsáveis por identificar o produto e seu respectivo valor.

Em seguida, ordenei os registros pela coluna `PRODUCT_VAL` em ordem decrescente para que os produtos com maior valor aparecessem primeiro. Por fim, utilizei a cláusula `LIMIT 10` para retornar apenas os 10 produtos mais caros da empresa, conforme solicitado no desafio.

```
SELECT PRODUCT_NAME, PRODUCT_VAL
FROM `loqbox-challenge`.data_product
ORDER BY PRODUCT_VAL DESC
LIMIT 10;
```

Resultado:

PRODUCT_NAME	PRODUCT_VAL
Whisky Escoces THE MACALLAN Ruby Garrafa ...	741.99
Whisky Escoces JOHNNIE WALKER Blue Label ...	735.90
Cafeteira Expresso 3 CORACOES Tres Modo V...	499.00
Vinho Portugues Tinto Vintage QUINTA DO CR...	445.90
Escova Dental Eletrica ORAL B D34 Profession...	399.90
Champagne Rose VEUVE CLICQUOT PONSARD...	366.90
Champagne Frances Brut Imperial MOET Rose ...	359.90
Conjunto de Panelas Allegra em Inox TRAMON...	359.00
Whisky Escoces CHIVAS REGAL 18 Anos Garra...	329.90
Champagne Frances Brut Imperial MOET & CH...	315.90

Questão 2

Pergunta: What sections do the 'BEBIDAS' and 'PADARIA' departments have?

Para responder essa pergunta, utilizei a tabela `data_product`, que contém as informações de departamentos e seções dos produtos. Como o objetivo era identificar quais seções pertenciam aos departamentos BEBIDAS e PADARIA, apliquei um filtro para retornar apenas os registros desses dois departamentos.

Utilizei a cláusula `DISTINCT` para eliminar possíveis duplicidades e garantir que cada seção fosse exibida apenas uma vez. Por fim, ordenei o resultado pelo nome do departamento para facilitar a leitura e a visualização das informações retornadas pela consulta.

```
SELECT DISTINCT DEP_NAME, SECTION_NAME
FROM `looqbox-challenge`.data_product
WHERE DEP_NAME IN ('BEBIDAS', 'PADARIA')
ORDER BY DEP_NAME;
```

Resultado:

DEP_NAME	SECTION_NAME
BEBIDAS	BEBIDAS
BEBIDAS	CERVEJAS
BEBIDAS	REFRESCOS
BEBIDAS	VINHOS
PADARIA	DOCES-E-SOBREMESAS
PADARIA	GESTANTE
PADARIA	PADARIA
PADARIA	QUEIJOS-E-FRIOS

Questão 3

Pergunta: What was the total sale of products (in \$) of each Business Area in the first quarter of 2019?

Para responder essa pergunta, optei por dividir a solução em duas etapas utilizando uma subquery. A primeira etapa teve como objetivo calcular o valor total vendido por registro. Para isso, selecionei a coluna `BUSINESS_NAME` e calculei uma nova coluna chamada `SALES`, multiplicando `SALES_VALUE` por `(SALES_QTY)`.

Em seguida, utilizei a tabela `data_product_sales` como base e realizei um `LEFT JOIN` com a tabela `data_store_cad` através da coluna `STORE_CODE`, permitindo relacionar as vendas às respectivas áreas de negócio (`BUSINESS_NAME`).

Após realizar o relacionamento entre as tabelas, apliquei um filtro para considerar apenas as vendas realizadas durante o primeiro trimestre de 2019. Para construir

corretamente esse filtro de período, utilizei apoio da IA para validar a melhor abordagem em SQL.

Na etapa final, utilizei a subquery como fonte de dados e agrupei os resultados por BUSINESS_NAME, somando a coluna SALES para obter o valor total vendido por cada área de negócio durante o período solicitado.

```
SELECT BUSINESS_NAME AS Business_Area, SUM(SALES) AS SALES$ FROM
(SELECT BUSINESS_NAME, (SALES_VALUE * SALES_QTY) AS SALES
FROM `looqbox-challenge`.data_product_sales sales
LEFT JOIN `looqbox-challenge`.data_store_cad store ON sales.STORE_CODE =
store.STORE_CODE
WHERE QUARTER(sales.DATE)=1 AND YEAR(sales.DATE)=2019) AS A
GROUP BY BUSINESS_NAME;
```

Resultado:

Business_Area	SALES\$
Varejo	16844042945.47
Farma	14002255081.83
Atacado	16597963597.16
Posto	5347998471.12
Proximidade	16567856155.16

Cases

Case 1

Inicialmente, criei a conexão com o banco de dados. Em seguida, desenvolvi a função `retrieve_data`, responsável por retornar os dados da tabela `data_product_sales` a partir dos parâmetros informados pelo usuário.

Durante o desenvolvimento, pensei em como tornar a função mais amigável para usuários não técnicos. Por isso, implementei uma etapa de validação e padronização das datas, permitindo que diferentes formatos fossem convertidos para o padrão utilizado na consulta SQL.

Após tratar os parâmetros, a função monta dinamicamente a query utilizando `product_code`, `store_code` e o intervalo de datas informado, retornando os registros correspondentes. Para a implementação da correção automática das datas, conexão com o banco de dados no python e para os comentários do código, utilizei apoio pontual de IA.

Código:

```
# 1. Configurações de acesso ao banco de dados
usuario = 'looqbox-challenge'
senha = 'looq-challenge'
```

```

host = '35.199.115.174'
porta = '3306'
banco = 'looqbox-challenge'
string_conexao =
f"mysql+mysqlconnector://{usuario}:{senha}@{host}:{porta}/{banco}"
engine = create_engine(string_conexao)

# 2. Função para corrigir as datas e montar a query
def retrieve_data(product_code:int, store_code:int, date:list[str]):
    datas_corrigidas = []

    # 1. LOOP PARA CORREÇÃO AUTOMÁTICA
    for d in date:
        try:
            data_objeto = parse(d, dayfirst=True)

            data_formatada = data_objeto.strftime("%Y-%m-%d")
            datas_corrigidas.append(data_formatada)

        except (ValueError, TypeError):
            # Se o texto for algo completamente ilegível (ex:
            "texto_qualquer")
            raise ValueError(f"Não foi possível entender a data: '{d}'.
            Digite uma data válida.")

    # 2. MONTAGEM DA QUERY
    query = (f"""SELECT *
                FROM
                    data_product_sales
                WHERE
                    PRODUCT_CODE = {product_code} AND
                    STORE_CODE = {store_code} AND
                    DATE BETWEEN '{datas_corrigidas[0]}' AND
                    '{datas_corrigidas[1]}' """)
    return pd.read_sql(query, con=engine)

my_data = retrieve_data(10, 10, ['01/01/2019', '2019-01-31'])

# 4. Fechar a conexão
engine.dispose()

```

Resultado:

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	10	10	2019-01-01	2386.15	65.0
1	10	10	2019-01-02	4368.49	119.0
2	10	10	2019-01-03	3854.55	105.0
3	10	10	2019-01-04	3671.00	100.0
4	10	10	2019-01-05	3010.22	82.0

Case 2

Assim como no Case 1, iniciei criando a conexão com o banco de dados e executando as duas consultas fornecidas pelo desafio, armazenando os resultados em DataFrames do Pandas. Em seguida, utilizei o método `head()` para entender a estrutura dos dados e validar se as informações estavam sendo carregadas corretamente.

Ao iniciar o filtro do período solicitado pelo desafio (01/10/2019 a 31/12/2019), encontrei um erro relacionado ao formato da coluna `DATE`. Para resolver o problema, converti a coluna para o tipo `datetime` utilizando o Pandas, permitindo que o filtro fosse aplicado corretamente.

Decidi começar as transformações pela tabela `data_store_sales`, pois precisava consolidar as informações por `STORE_CODE` antes de realizar o merge. Para isso, agrupei os dados por loja, somando os valores de `SALES_VALUE` e `SALES_QTY`. Essa etapa também garantiu que não existissem registros duplicados de `STORE_CODE` no momento da junção das tabelas.

Antes do merge, foi necessário calcular a coluna `TM`. Como o desafio não informava explicitamente o significado dessa métrica, pesquisei e utilizei apoio da IA para entender o contexto. A interpretação sugerida foi que `TM` representava Ticket Médio, calculado pela divisão entre `SALES_VALUE` e `SALES_QTY`. Após o cálculo, arredondei os valores para duas casas decimais.

Com os dados preparados, realizei o merge utilizando `STORE_CODE` como chave de relacionamento. Por fim, executei alguns tratamentos para adequar o resultado ao formato solicitado, incluindo renomeação de colunas, seleção e ordenação dos campos finais. Também utilizei apoio da IA para converter o separador decimal de ponto para vírgula na coluna `TM`, reproduzindo o padrão apresentado no exemplo do desafio.

Código:

```
# 1. Configurações de acesso ao banco de dados
usuario = 'looqbox-challenge'
senha = 'looq-challenge'
```

```

host = '35.199.115.174'
porta = '3306'
banco = 'looqbox-challenge'
string_conexao =
f"mysql+mysqlconnector://{usuario}:{senha}@{host}:{porta}/{banco}"
engine = create_engine(string_conexao)

# 2. Consultas SQL para ler as tabelas
query1 = """SELECT
    STORE_CODE,
    STORE_NAME,
    START_DATE,
    END_DATE,
    BUSINESS_NAME,
    BUSINESS_CODE
FROM data_store_cad"""

query2 = """SELECT
    STORE_CODE,
    DATE,
    SALES_VALUE,
    SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'"""

# 3. Ler as tabelas usando as consultas SQL e armazenar em DataFrames do
Pandas
df1 = pd.read_sql(query1, con=engine)
df2 = pd.read_sql(query2, con=engine)

# 4. Fechar a conexão
engine.dispose()

# 5. transformar a coluna 'DATE' para o formato datetime e filtrar as datas
df2['DATE'] = pd.to_datetime(df2['DATE'])
df2 = df2[(df2['DATE'] >= '2019-10-01') & (df2['DATE'] <= '2019-12-31')]

# Agrupar por STORE_CODE e calcular o Ticket Médio
df2 = df2.groupby('STORE_CODE').agg({'SALES_VALUE': 'sum',
    'SALES_QTY': 'sum'}).reset_index()
df2['TM'] = df2['SALES_VALUE'] / df2['SALES_QTY']
df2['TM'] = df2['TM'].round(2)

# 6. Fazer o merge entre os DataFrames df1 e df2 usando a coluna
'STORE_CODE' como chave
tm = df1.merge(df2,
                left_on='STORE_CODE',
                right_on='STORE_CODE',
                how='left')

# Transformações finais para exibição do resultado
tm = tm.rename(columns={'STORE_NAME': 'Loja', 'BUSINESS_NAME': 'Categoria'})
tm = tm[['Loja', 'Categoria', 'TM']]
tm = tm.sort_values(by='Loja')

```

```
tm['TM'] = tm['TM'].map('{:.2f}'.format).str.replace('.', ', ', regex=False)
#Feito com IA
```

Resultado:

	Loja	Categoria	TM
13	Bahia	Atacado	15,39
17	Bangkok	Posto	13,67
7	Belem	Proximidade	15,37
5	Berlin	Proximidade	15,39
14	Buenos Aires	Atacado	15,39
1	Chicago	Varejo	15,53
12	Dubai	Atacado	15,39
9	Hong Kong	Farma	26,35
8	London	Farma	28,99
11	Madri	Farma	29,03
18	Miami	Posto	13,67
6	New York	Proximidade	15,39
4	Paris	Proximidade	15,39
10	Rio de Janeiro	Farma	29,59
2	Roma	Varejo	15,39
15	Salvador	Atacado	15,39
0	Sao Paulo	Varejo	15,39
16	Sidney	Posto	13,67
3	Tokio	Varejo	15,39
19	Vancouver	Posto	13,67

Case 3

Iniciei realizando a conexão com o banco de dados e executando uma consulta para buscar todas as colunas da tabela IMDB_movies. Antes de criar os gráficos, explorei os dados para entender quais informações poderiam gerar análises interessantes e quais visualizações seriam mais adequadas para cada contexto.

Gráfico 1 – Evolução da nota média dos filmes ao longo dos anos

O primeiro gráfico que decidi criar foi um gráfico de linha mostrando a evolução da nota média dos filmes ao longo dos anos. Escolhi esse tipo de visualização por ser uma

das formas mais adequadas de representar tendências temporais e facilitar a identificação de mudanças ao longo do período analisado.

Para construir o gráfico, agrupei os dados por ano e calculei a média da coluna Rating. Inicialmente considerei utilizar a receita média dos filmes, porém optei pela nota média por entender que a receita poderia sofrer influência da inflação ao longo dos anos, o que poderia gerar interpretações equivocadas caso esse fator não fosse tratado adequadamente.

Utilizei apoio da IA apenas para auxiliar na configuração de alguns parâmetros do Matplotlib, pois não é uma biblioteca que utilizo com frequência no meu dia a dia profissional.

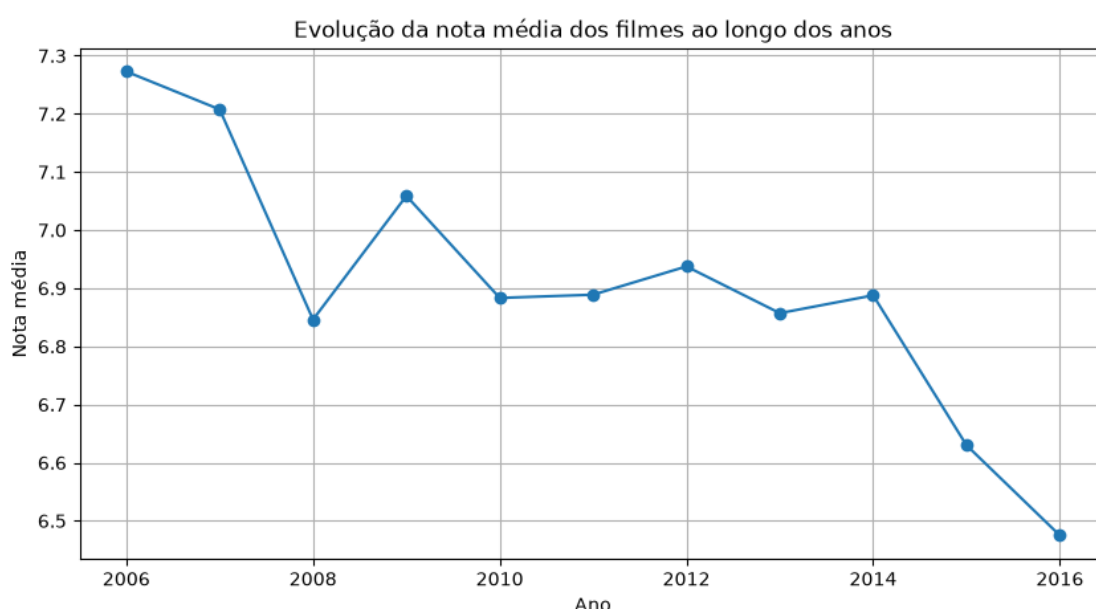


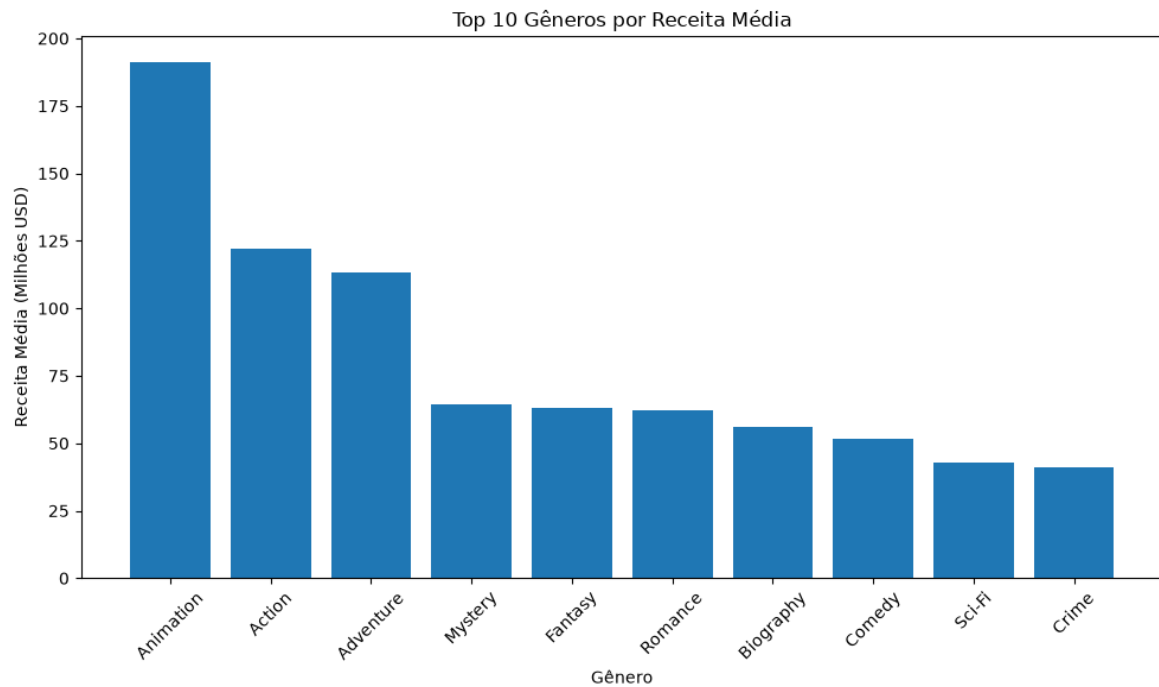
Gráfico 2 – Top 10 gêneros por receita

Para o segundo gráfico, decidi analisar quais gêneros apresentaram as maiores receitas. Como alguns filmes possuíam mais de um gênero cadastrado, optei por utilizar apenas o gênero principal de cada registro. Para essa etapa, utilizei apoio da IA para identificar uma forma eficiente de extrair o primeiro gênero da coluna.

Após isso, agrupei os dados por gênero, calculei a receita média e ordenei os resultados de forma decrescente. Em seguida, selecionei os 10 gêneros com maior receita utilizando o método `head()` e construí um gráfico de barras.

Escolhi esse tipo de visualização porque ele permite comparar categorias de forma rápida e intuitiva, facilitando a identificação dos gêneros com melhor desempenho

financeiro. Assim como no primeiro gráfico, utilizei apoio da IA apenas para auxiliar na configuração dos parâmetros do Matplotlib.



Código:

```
# 1. Configurações de acesso ao banco de dados
usuario = 'looqbox-challenge'
senha = 'looq-challenge'
host = '35.199.115.174'
porta = '3306'
banco = 'looqbox-challenge'
string_conexao =
f"mysql+mysqlconnector://{usuario}:{senha}@{host}:{porta}/{banco}"
engine = create_engine(string_conexao)

# 2. Criando query para ler a tabela IMDB_movies
query = """SELECT* FROM IMDB_movies"""

df = pd.read_sql(query, con=engine)

# 3. Fechar a conexão
engine.dispose()
# Criação de média para gráfico de linha
media_ano = df.groupby('Year').agg({'Rating':'mean'}).reset_index()
media_ano.head()
# Criação do gráfico de linha
plt.figure(figsize=(10,5))
plt.plot(
    media_ano["Year"],
    media_ano["Rating"],
    marker="o"
```

```

)
plt.title("Evolução da nota média dos filmes ao longo dos anos")
plt.xlabel("Ano")
plt.ylabel("Nota média")
plt.grid(True)

# Buscando o gênero principal de cada filme (o primeiro da lista de
gêneros)
df["Main_Genre"] = df["Genre"].str.split(",").str[0]

# Criação de média para gráfico de barras
receita_diretor =
df.groupby('Main_Genre').agg({'RevenueMillions':'mean'}).reset_index().sort
_values("RevenueMillions", ascending=False)
receita_diretor.head()
# Selecionando os 10 gêneros com maior receita média
top10 = receita_diretor.head(10)

# Criação do gráfico de barras
plt.figure(figsize=(10,6))

plt.bar(
    top10["Main_Genre"],
    top10["RevenueMillions"]
)
plt.title("Top 10 Gêneros por Receita Média")
plt.xlabel("Gênero")
plt.ylabel("Receita Média (Milhões USD)")
plt.xticks(rotation=45)
plt.tight_layout()

```

Uso de IA

Durante a realização do desafio, utilizei ferramentas de Inteligência Artificial como apoio pontual para esclarecer dúvidas específicas e acelerar algumas etapas de desenvolvimento. A IA foi utilizada principalmente para:

- Ajudar na conexão com o banco de dados
- Auxiliar na implementação da validação e padronização de datas no Case 1;
- Revisar e gerar comentários de código em alguns trechos da solução;
- Ajudar na interpretação da sigla "TM" no Case 2, sugerindo que poderia representar Ticket Médio;
- Auxiliar na formatação final dos valores da coluna TM para o padrão apresentado no desafio;
- Esclarecer dúvidas sobre filtros de datas em SQL;
- Apoiar a extração do gênero principal da coluna Genre no Case 3;

- Auxiliar na configuração de alguns parâmetros de visualização utilizando Matplotlib.
- Revisar os erros ortográficos no PDF

Toda a lógica de negócio, modelagem das consultas SQL, definição das transformações, estruturação das soluções, interpretação dos dados, criação dos gráficos e validação dos resultados foram realizadas por mim. A IA foi utilizada como uma ferramenta de apoio técnico, semelhante ao uso de documentação, fóruns ou materiais de consulta, e não como substituta do raciocínio utilizado na resolução do desafio.